

Introduction to JSP

Java Server Pages (JSP) is a server-side programming technology that creates dynamic web applications, platform-independent method for building Web-based applications.

It allows developers to embed Java code directly into HTML pages and it makes web development more efficient. JSP have access to the entire family of Java APIs, including the JDBC API to access enterprise databases.

JSP is an advanced version of Servlets. It provides enhanced capabilities for building scalable and platform-independent web pages.

JSP - Lifecycle

A JSP life cycle is defined as the process from its creation till the destruction. This is similar to a servlet life cycle with an additional step which is required to compile a JSP into servlet.

The following are the Steps followed by a JSP -

1. Translate the JSP into Servlet code
2. Compile the servlet into ByteCode
3. Load the Servlet class
4. Create the Servlet instance
5. Call the `jspInit()` method.
6. Call the `_jspService()` method.
7. Call the `jspDestroy()` method.

The Seven major phases of a JSP life cycle are very similar to the Servlet Life Cycle. The Seven phases have been described below -

JSP Compilation

When a browser asks for a JSP, the JSP engine first checks to see whether it needs to compile the page. If the page has never been compiled, or if the JSP has been modified since it was last compiled, the JSP engine compiles the page.

The compilation process involves three steps -

1. Parsing the JSP.
2. Converting the JSP into a servlet.
3. Compiling the servlet.

JSP Initialization

When a container loads a JSP it invokes the `jspInit()` method before servicing any requests. If you need to perform JSP-specific initialization, override the `jspInit()` method -

```
public void jspInit()
{
    // Initialization code...
}
```

Typically, initialization is performed only once and as with the `Servlet.init()` method, you generally initialize database connections, open files, and create lookup tables in the `jspInit` method.



JSP Execution

This phase of the JSP life cycle represents all interactions with requests until the JSP is destroyed.

Whenever a browser requests a JSP and the page has been loaded and initialized, the JSP engine invokes the `_jspService()` method in the JSP.

The `_jspService()` method takes an `HttpServletRequest` and an `HttpServletResponse` as its parameters as follows -

```
void _jspService(HttpServletRequest request,  
HttpServletResponse response)  
{  
    // Service handling code... }
```

The `_jspService()` method of a JSP is invoked on request basis. This is responsible for generating the response for that request and this method is also responsible for generating responses to all seven of the HTTP methods, i.e, **GET**, **POST**, **DELETE**, etc.



JSP Cleanup

The destruction phase of the JSP life cycle represents when a JSP is being removed from use by a container.

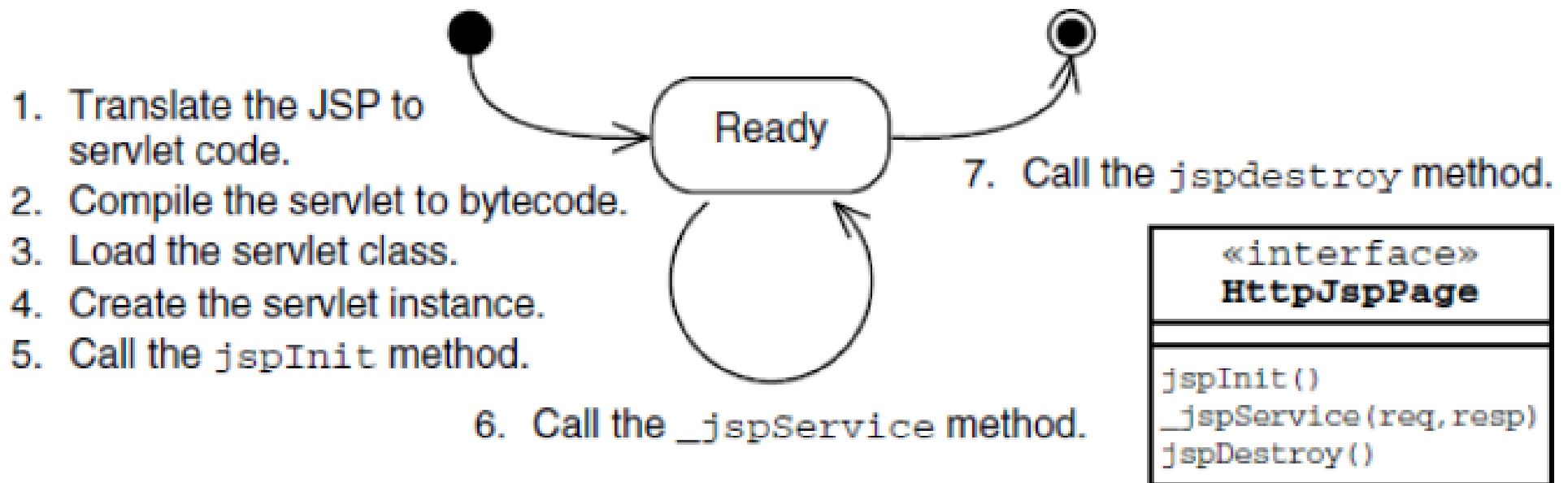
The `jspDestroy()` method is the JSP equivalent of the `destroy` method for servlets. Override `jspDestroy` when you need to perform any cleanup, such as releasing database connections or closing open files.

The `jspDestroy()` method has the following form

```
-  
public void jspDestroy()  
{  
    // Your cleanup code goes here.  
}
```

How a JSP Page is Processed

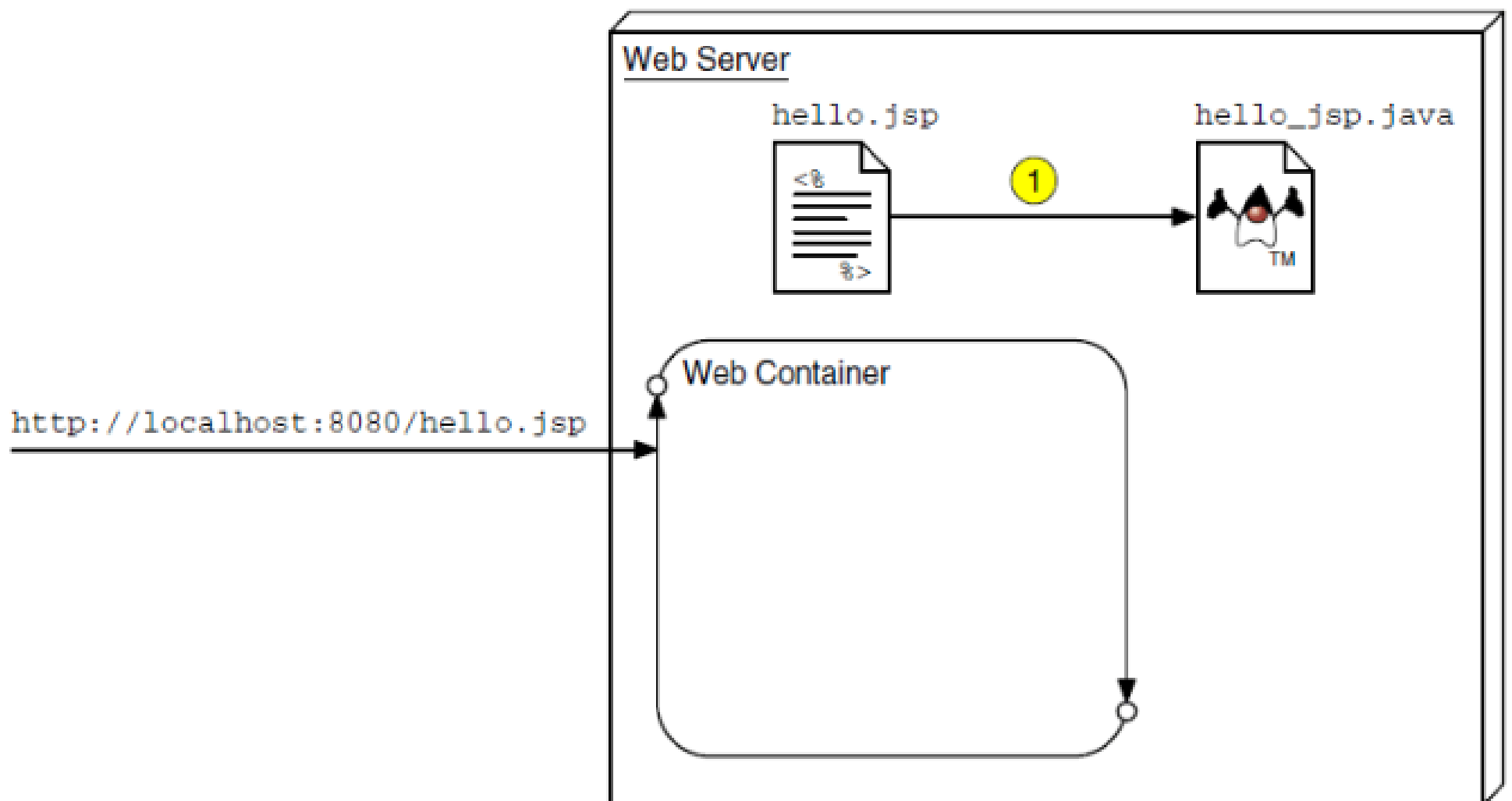
A JSP page is essentially source code, and must be converted into a servlet before it can service requests.



Steps of JSP Page Processing

JSP Page Translation

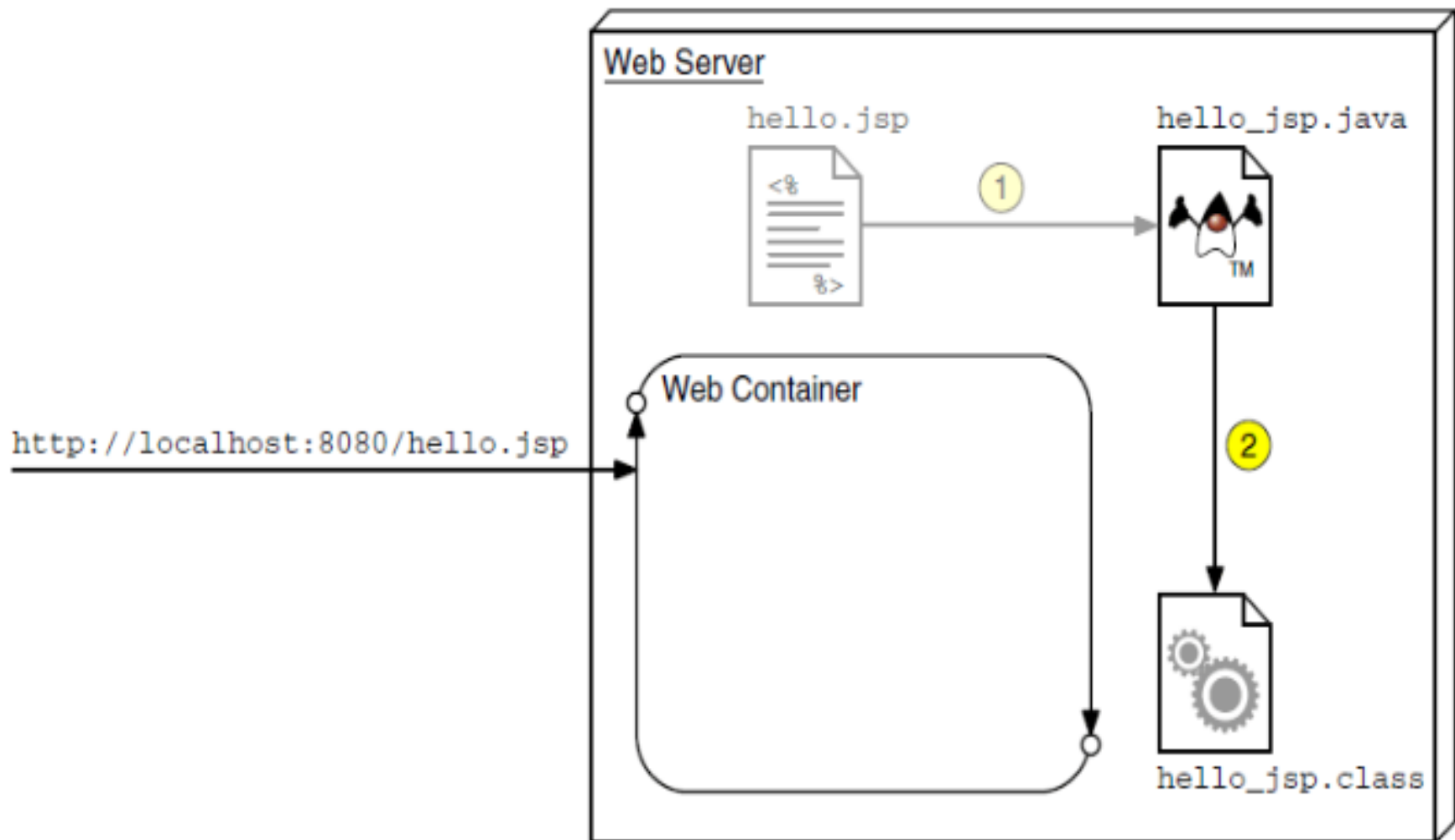
In the first step, the web container translates the JSP file into a Java source file that contains a servlet class definition.



JSP Page Processing: Translation Step

JSP Page Compilation

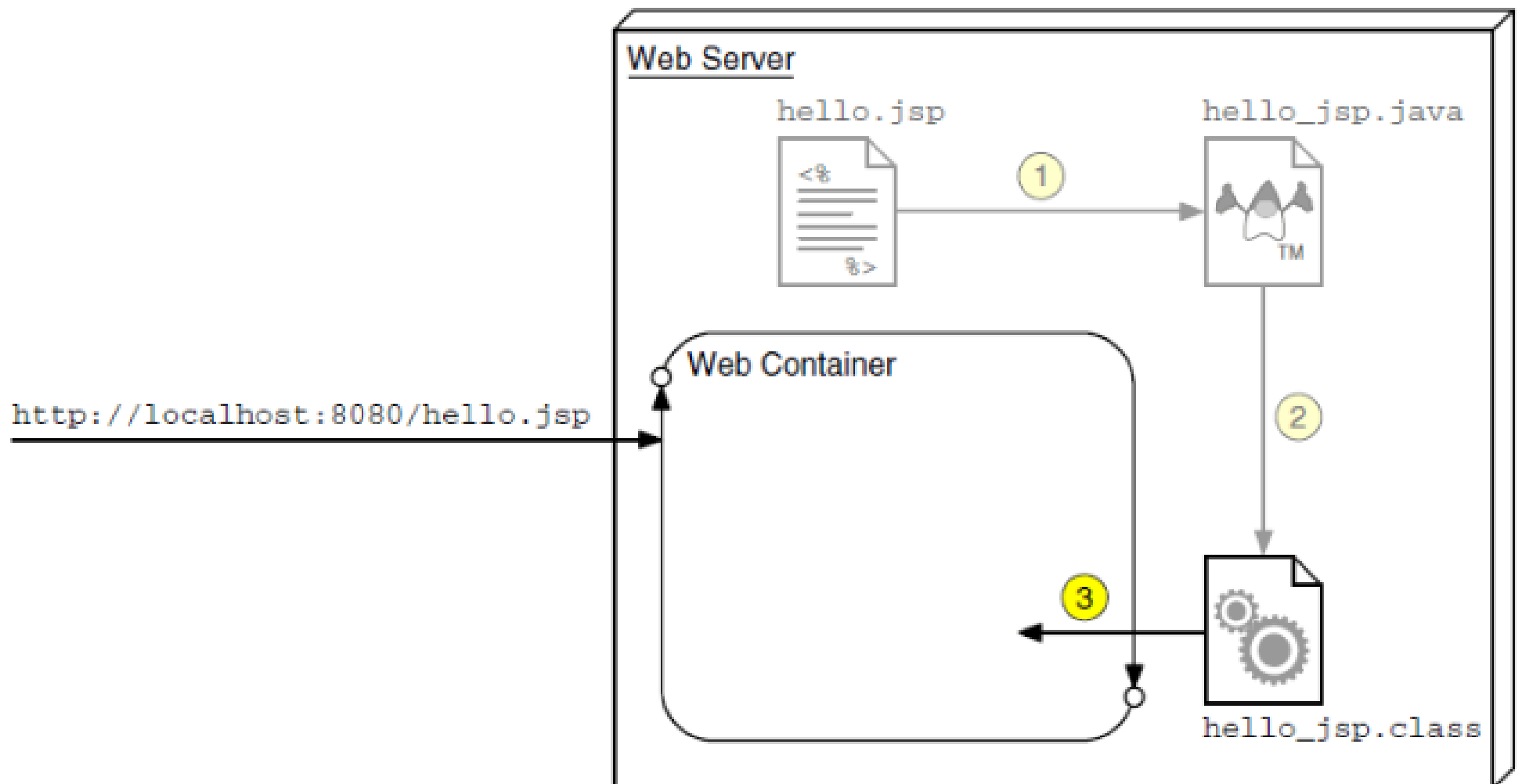
In the second step, the web container compiles the servlet source code into a Java class file.



JSP Page Processing: Compilation Step

JSP Page Class Loading

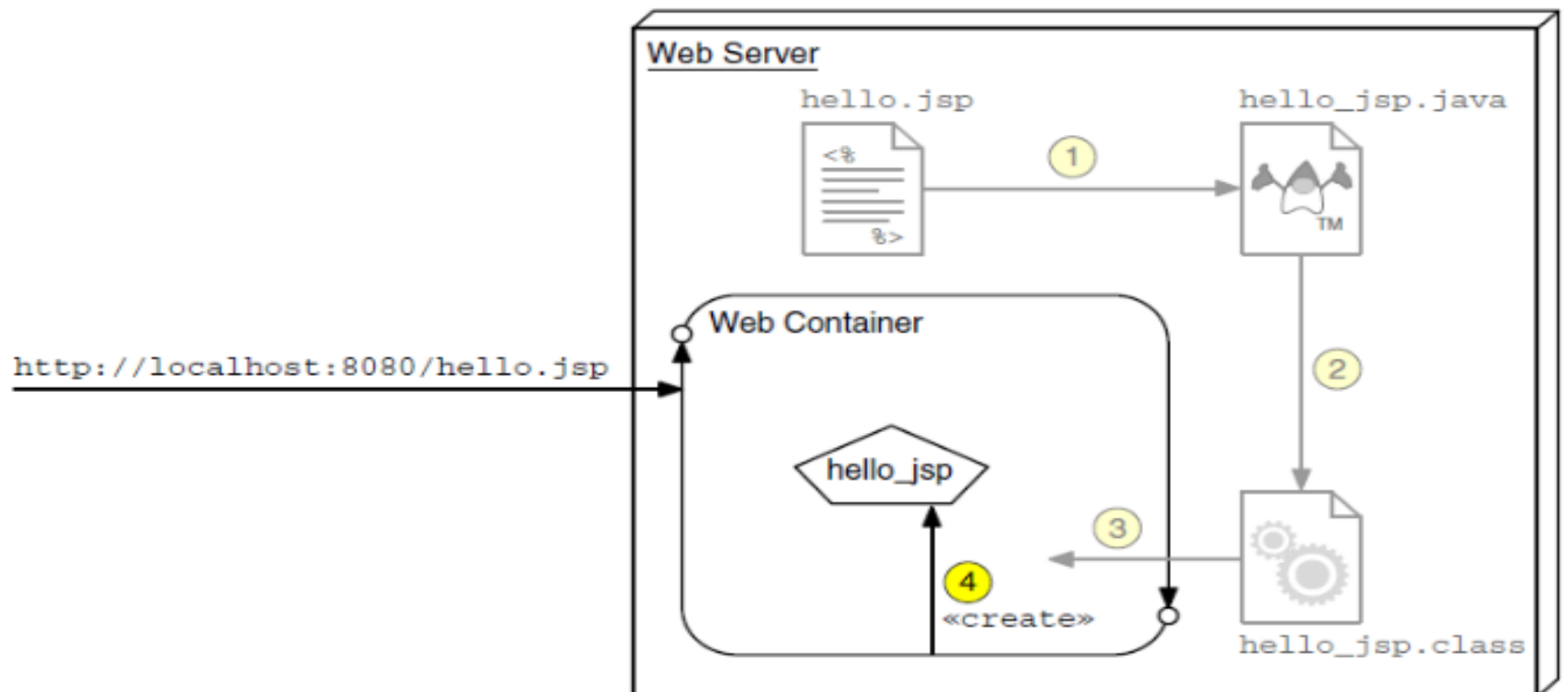
In the third step, the servlet class bytecode is loaded into the web container's JVM software using a classloader.



JSP Page Processing: Load Class Step

JSP Page Servlet Instance

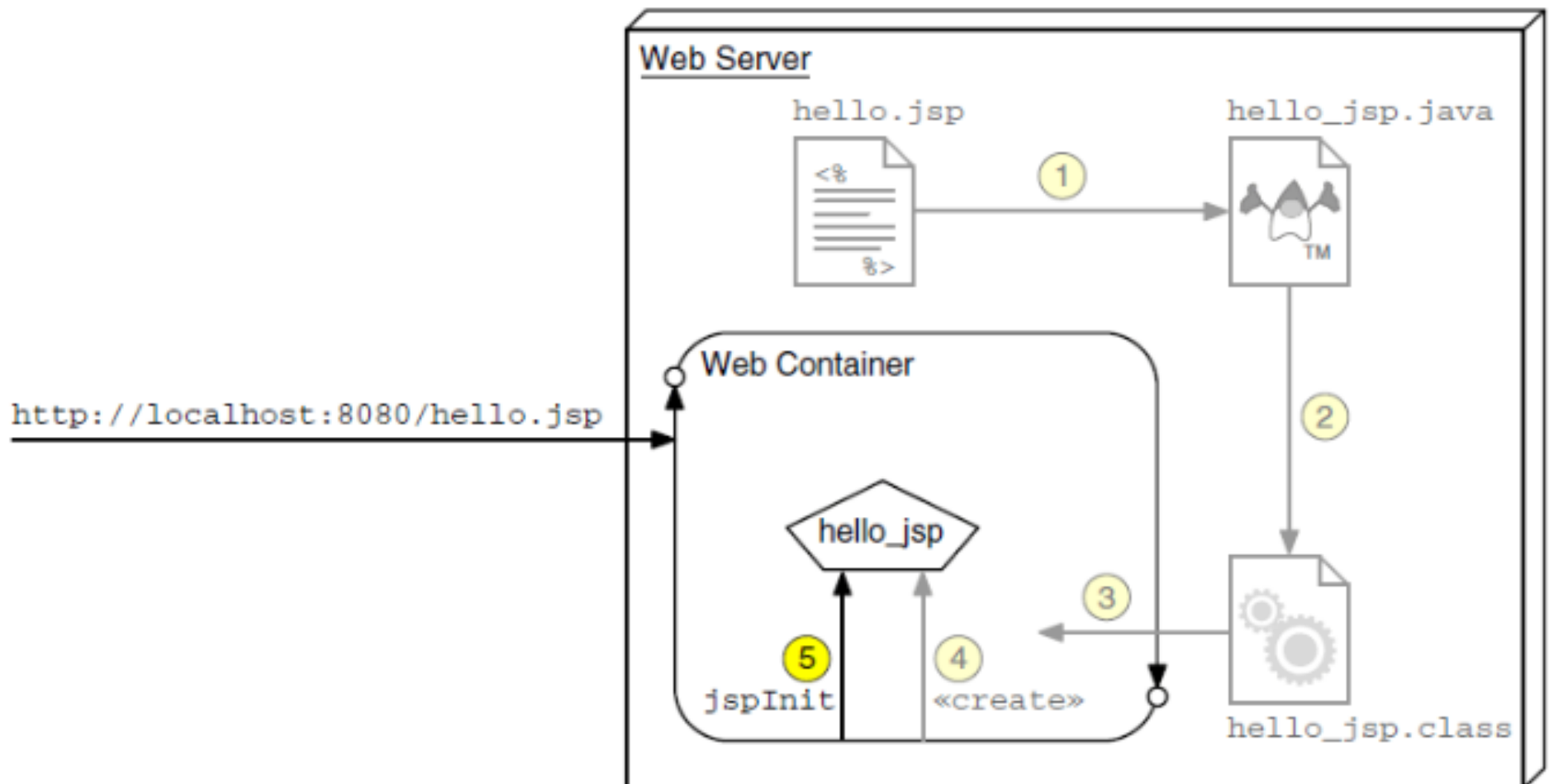
In the fourth step, the web container creates an instance of the servlet class.



JSP Page Processing: Instance Creation Step

JSP Page Initialization

In the fifth step, the web container initializes the servlet by calling the `jspInit` method.

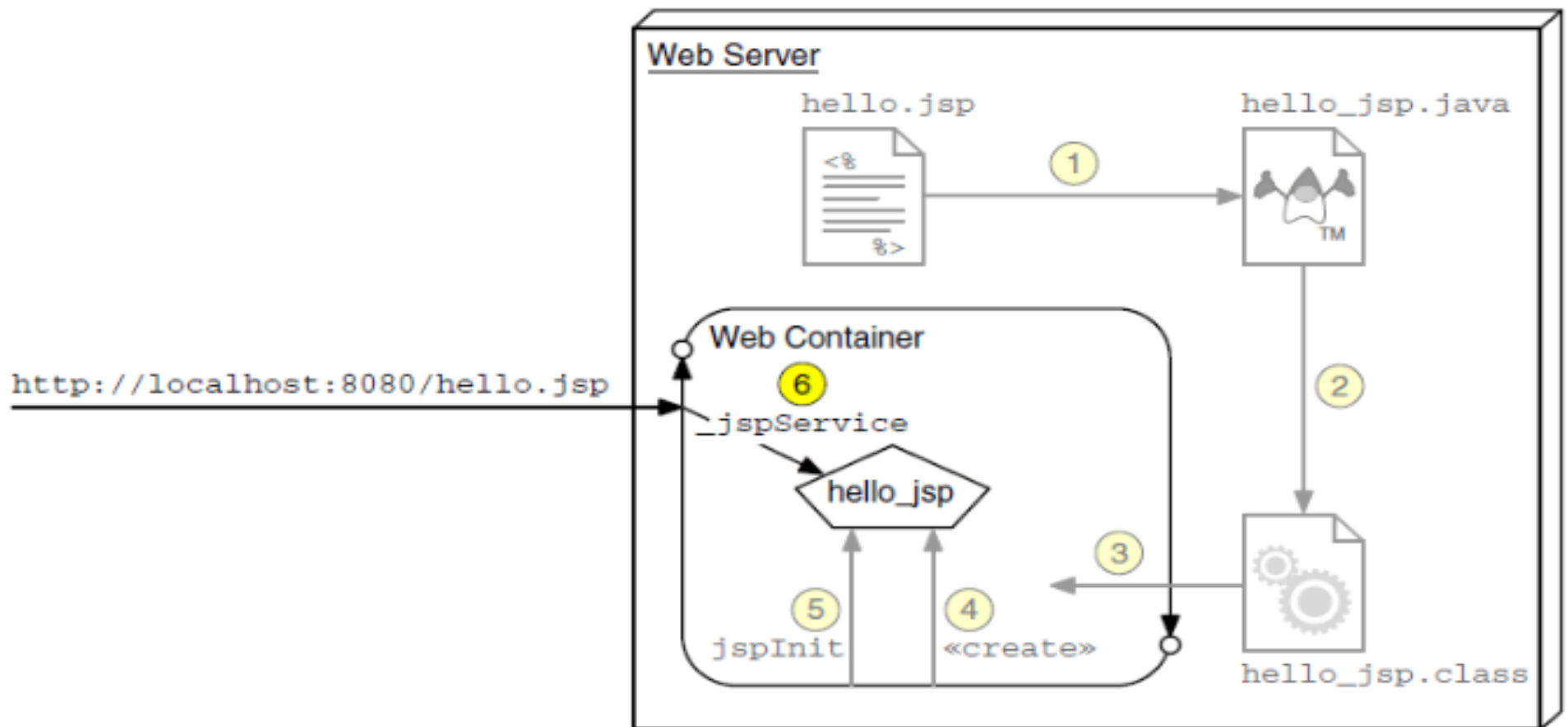


JSP Page Processing: Initialization Step

Actually, the container calls the `init(SC)` method, but your JSP page servlet inherits an `init` method that forwards the call to the `jspInit` method.

JSP Page Service

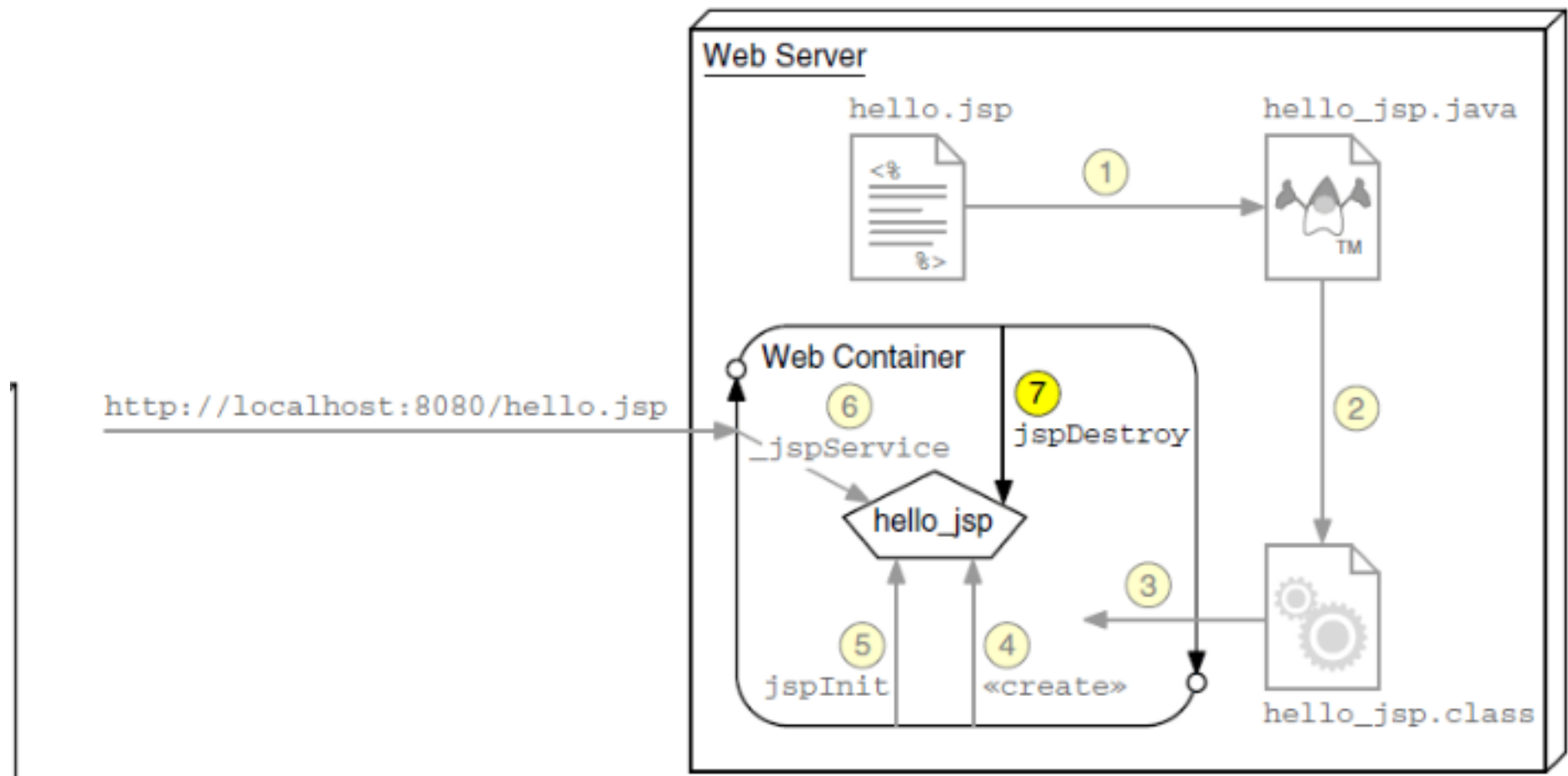
The initialized servlet can now service requests. With each request, the web container can call the `_jspService` method for the converted JSP page.



JSP Page Processing: Request Handling Step

JSP Page Destroyed

When the web container removes the JSP servlet instance from service, it first calls the `jspDestroy` method to allow the JSP page to perform any required clean up.



JSP Page Processing: Destroy Step

Note The process of translating, compiling, loading, and initializing is performed every time the JSP file changes within the web container's deployment environment. This makes developing JSP pages easier.

Writing JSP Scripting Elements

JSP scripting elements are embedded with the `<% %>` tags and are processed by the JSP engine during translation of the JSP page. Any other text in the JSP page is considered part of the response and is copied verbatim to the HTTP response stream that is sent to the web browser.

`<html> <%--scripting element --%></html>`

<i>Scripting Element</i>	<i>Example</i>
<i>Comment</i>	<code><%-- comment --%></code>
<i>Directive</i>	<code><%@ directive %></code>
<i>Declaration</i>	<code><%!declaration %></code>
<i>Scriptlet</i>	<code><% code %></code>
<i>Expression</i>	<code><%= expression %></code>



The Scriptlet

A scriptlet can contain any number of JAVA language statements, variable or method declarations, or expressions that are valid in the page scripting language.

Following is the syntax of Scriptlet –

```
<% code fragment %>
```

You can write the XML equivalent of the above syntax as follows –

```
<jsp:scriptlet>  
    code fragment  
</jsp:scriptlet>
```



Any text, HTML tags, or JSP elements you write must be outside the scriptlet. Following is the simple and first example for JSP -

```
<html>
```

```
  <head><title>Hello World</title></head>
```

```
  <body>
```

```
    Hello World!<br/>
```

```
    <%
```

```
      out.println("Your IP address is " +  
request.getRemoteAddr());
```

```
    %>
```

```
  </body>
```

```
</html>
```

JSP Declarations

A declaration declares one or more variables or methods that you can use in Java code later in the JSP file. You must declare the variable or method before you use it in the JSP file.

Following is the syntax for JSP Declarations -

```
<%! declaration; [ declaration; ]+ ... %>
```

You can write the XML equivalent of the above syntax as follows -

```
<jsp:declaration>  
    code fragment  
</jsp:declaration>
```

Following is an example for JSP Declarations -

```
<%! int i = 0; %>  
<%! int a, b, c; %>  
<%! Circle a = new Circle(2.0); %>
```

JSP Expression

A JSP expression element contains a scripting language expression that is evaluated, converted to a String, and inserted where the expression appears in the JSP file.

Because the value of an expression is converted to a String, you can use an expression within a line of text, whether or not it is tagged with HTML, in a JSP file.

The expression element can contain any expression that is valid according to the Java Language Specification but you cannot use a semicolon to end an expression.

Following is the syntax of JSP Expression -

```
<%= expression %>
```



You can write the XML equivalent of the above syntax as follows –

```
<jsp:expression>  
    expression  
</jsp:expression>
```

Following example shows a JSP Expression –

```
<html>  
    <head><title>A Comment Test</title></head>  
  
    <body>  
        <p>Today's date: <%= (new  
                                java.util.Date()).toLocaleString()%> </p>  
    </body>  
</html>
```


JSP Comments

JSP comment marks text or statements that the JSP container should ignore. A JSP comment is useful when you want to hide or "comment out", a part of your JSP page.

Following is the syntax of the JSP comments -

```
<%-- This is JSP comment --%>
```

Following example shows the JSP Comments -

```
<html>
  <head><title>A Comment Test</title></head>
  <body>
    <h2>A Test of Comments</h2>
    <%-- This comment will not be visible in the page
source --%>
  </body>
</html>
```

JSP Directives

A JSP directive affects the overall structure of the servlet class. It usually has the following form –

`<%@ directive attribute="value" %>`

There are three types of directive tag –

1. `<%@ page ... %>`

Defines page-dependent attributes, such as scripting language, error page, and buffering requirements.

2. `<%@ include ... %>`

Includes a file during the translation phase.

3. `<%@ taglib ... %>`

Declares a tag library, containing custom actions, used in the page

JSP Actions

JSP actions use constructs in XML syntax to control the behavior of the servlet engine. You can dynamically insert a file, reuse JavaBeans components, forward the user to another page, or generate HTML for the Java plugin.

There is only one syntax for the Action element, as it conforms to the XML standard -

```
<jsp:action_name attribute="value" />
```

Action elements are basically predefined functions. Following table lists out the available JSP Actions –

1. `jsp:include`

Includes a file at the time the page is requested.

2. `jsp:useBean`

Finds or instantiates a JavaBean.

3. `jsp:setProperty`

Sets the property of a JavaBean.

4. `jsp:getProperty`

Inserts the property of a JavaBean into the output.

5. `jsp:forward`

Forwards the requester to a new page.

6. `jsp:plugin`

Generates browser-specific code that makes an OBJECT or EMBED tag for the Java plugin.

7. `jsp:element`

Defines XML elements dynamically.

8. `jsp:attribute`

Defines dynamically-defined XML element's attribute.

9. `jsp:body`

Defines dynamically-defined XML element's body.

10. `jsp:text`

Used to write template text in JSP pages and documents.

JSP Implicit Objects

Implicit Objects in JSP are the Java objects that the JSP Container makes available to the developers in each page and the developer can call them directly without being explicitly declared.

JSP Implicit Objects are also called **pre-defined variables**.

1. request

This is the `HttpServletRequest` object associated with the request.

2. response

This is the `HttpServletResponse` object associated with the response to the client.

3. out

This is the `PrintWriter` object used to send output to the client.



4. session

This is the HttpSession object associated with the request.

5. application

This is the ServletContext object associated with the application context.

6. config

This is the ServletConfig object associated with the page.

7. pageContext

This encapsulates use of server-specific features like higher performance JspWriters.

8. page

This is simply a synonym for this, and is used to call the methods defined by the translated servlet class.

9. Exception

The Exception object allows the exception data to be accessed by designated JSP.



request Object

The request object is an instance of a `javax.servlet.http.HttpServletRequest` object. Each time a client requests a page the JSP engine creates a new object to represent that request.

The request object provides methods to get the HTTP header information including form data, cookies, HTTP methods etc.



response Object

The response object is an instance of a `javax.servlet.http.HttpServletResponse` object. Just as the server creates the request object, it also creates an object to represent the response to the client.

The response object also defines the interfaces that deal with creating new HTTP headers. Through this object the JSP programmer can add new cookies or date stamps, HTTP status codes, etc.



The out Object

The out implicit object is an instance of a `javax.servlet.jsp.JspWriter` object and is used to send content in a response.

The initial JspWriter object is instantiated differently depending on whether the page is buffered or not. Buffering can be easily turned off by using the `buffered = 'false'` attribute of the page directive.

The JspWriter object contains most of the same methods as the `java.io.PrintWriter` class. However, JspWriter has some additional methods designed to deal with buffering. Unlike the PrintWriter object, JspWriter throws **IOExceptions**.



1. `out.print(dataType dt)`

Print a data type value

2. `out.println(dataType dt)`

Print a data type value then terminate the line with new line character.

3. `out.flush()`

Flush the stream.

The session Object

The session object is an instance of `javax.servlet.http.HttpSession` and behaves exactly the same way that session objects behave under Java Servlets.

The session object is used to track client session between client requests.

The application Object

The application object is direct wrapper around the **ServletContext** object for the generated Servlet and in reality an instance of a **javax.servlet.ServletContext** object.

This object is a representation of the JSP page through its entire lifecycle. This object is created when the JSP page is initialized and will be removed when the JSP page is removed by the **jspDestroy()** method.

By adding an attribute to application, you can ensure that all JSP files that make up your web application have access to it.



config Object

The config object is an instantiation of `javax.servlet.ServletConfig` and is a direct wrapper around the `ServletConfig` object for the generated servlet.

This object allows the JSP programmer access to the Servlet or JSP engine initialization parameters such as the paths or file locations etc.

The following config method is the only one you might ever use, and its usage is trivial -

```
config.getServletName();
```

This returns the servlet name, which is the string contained in the `<servlet-name>` element defined in the `WEB-INF\web.xml` file.

pageContext Object

The pageContext object is an instance of a `javax.servlet.jsp.PageContext` object. The pageContext object is used to represent the entire JSP page.

This object is intended as a means to access information about the page while avoiding most of the implementation details.

This object stores references to the request and response objects for each request. The application, config, session, and out objects are derived by accessing attributes of this object.

The pageContext object also contains information about the directives issued to the JSP page, including the buffering information, the `errorPageURL`, and page scope.



The `PageContext` class defines several fields, including `PAGE_SCOPE`, `REQUEST_SCOPE`, `SESSION_SCOPE`, and `APPLICATION_SCOPE`, which identify the four scopes. It also supports more than 40 methods, about half of which are inherited from the `javax.servlet.jsp.JspContext` class.

One of the important methods is `removeAttribute`. This method accepts either one or two arguments. For example, `pageContext.removeAttribute("attrName")` removes the attribute from all scopes, while the following code only removes it from the page scope –

```
pageContext.removeAttribute("attrName",  
PAGE_SCOPE);
```



page Object

This object is an actual reference to the instance of the page. It can be thought of as an object that represents the entire JSP page.

The page object is really a direct synonym for the `this` object.

exception Object

The exception object is a wrapper containing the exception thrown from the previous page. It is typically used to generate an appropriate response to the error condition.

JSP - Client Request

When a browser requests for a Webpage, it sends a lot of information to the web server. This information cannot be read directly because this information travels as a part of header of HTTP request.

Following are the important header information which comes from the browser.

1. Accept

This header specifies the MIME types that the browser or other clients can handle. Values of image/png or image/jpeg are the two most common possibilities.

2. Accept-Charset

This header specifies the character sets that the browser can use to display the information. For example, ISO-8859-1.



3. Accept-Encoding

This header specifies the types of encodings that the browser knows how to handle. Values of gzip or compress are the two most common possibilities.

4. Accept-Language

This header specifies the client's preferred languages in case the servlet can produce results in more than one language. For example en, en-us, ru, etc.

5. Authorization

This header is used by clients to identify themselves when accessing password-protected webpages.

6. Connection

This header indicates whether the client can handle persistent HTTP connections. Persistent connections permit the client or other browser to retrieve multiple files with a single request. A value of Keep-Alive means that persistent connections should be used.

7. Content-Length

This header is applicable only to POST requests and gives the size of the POST data in bytes.



8. Cookie

This header returns cookies to servers that previously sent them to the browser.

9. Host

This header specifies the host and port as given in the original URL.

10. If-Modified-Since

This header indicates that the client wants the page only if it has been changed after the specified date. The server sends a code, 304 which means Not Modified header if no newer result is available.

11. If-Unmodified-Since

This header is the reverse of If-Modified-Since; it specifies that the operation should succeed only if the document is older than the specified date.

12. Referer

This header indicates the URL of the referring webpages. For example, if you are at Webpage 1 and click on a link to Webpage 2, the URL of Webpage 1 is included in the Referer header when the browser requests Webpage 2.

13. User-Agent

This header identifies the browser or other client making the request and can be used to return different content to different types of browsers.

The `HttpServletRequest` Object

The request object is an instance of a `javax.servlet.http.HttpServletRequest` object. Each time a client requests a page, the JSP engine creates a new object to represent that request.

The request object provides methods to get HTTP header information including **form data**, **cookies**, **HTTP methods**, etc.

Following table lists out the important methods that can be used to read HTTP header in your JSP program. These methods are available with *HttpServletRequest* object which represents client request to webserver.

1. `Cookie[] getCookies()`

Returns an array containing all of the Cookie objects the client sent with this request.



2. `Enumeration getAttributeNames()`

Returns an Enumeration containing the names of the attributes available to this request.

3. `Enumeration getHeaderNames()`

Returns an enumeration of all the header names this request contains.

4. `Enumeration getParameterNames()`

Returns an enumeration of String objects containing the names of the parameters contained in this request.

5. `HttpSession getSession()`

Returns the current session associated with the this request, or if the request does not have a session, creates one.

6. `HttpSession getSession(boolean create)`

Returns the current HttpSession associated with the this request or, if there is no current session and create is true, returns a new session.

7. Locale getLocale()

Returns the preferred Locale that the client will accept content in, based on the Accept-Language header.



8. Object getAttribute(String name)

Returns the value of the named attribute as an Object, or null if no attribute of the given name exists.

9. ServletInputStream getInputStream()

Retrieves the body of the request as binary data using a ServletInputStream.

10. String getAuthType()

Returns the name of the authentication scheme used to protect the servlet, for example, "BASIC" or "SSL," or null if the JSP was not protected.

11. String getCharacterEncoding()

Returns the name of the character encoding used in the body of this request.

12. String getContentType()

Returns the MIME type of the body of the request, or null if the type is not known.

13. String getContextPath()

Returns the portion of the request URI that indicates the context of the request.

14. String getHeader(String name)

Returns the value of the specified request header as a String.

15. String getMethod()

Returns the name of the HTTP method with which this request was made, for example, GET, POST, or PUT.

16. String getParameter(String name)

Returns the value of a request parameter as a String, or null if the parameter does not exist.

17. String getPathInfo()

Returns any extra path information associated with the URL the client sent when it made this request.

18. String getProtocol()

Returns the name and version of the protocol the request uses.

19. String getQueryString()

Returns the query string that is contained in the request URL after the path.

20. String getRemoteAddr()

Returns the Internet Protocol (IP) address of the client that sent the request.

21. String getRemoteHost()

Returns the fully qualified name of the client that sent the request.

22. String getRemoteUser()

Returns the login of the user making this request, if the user has been authenticated, or null if the user has not been authenticated.

23. String getRequestURI()

Returns the part of this request's URL from the protocol name up to the query string in the first line of the HTTP request.

24. String getRequestedSessionId()

Returns the session ID specified by the client.

25. `String getServletPath()`

Returns the part of this request's URL that calls the JSP.

26. `String[] getParameterValues(String name)`

Returns an array of String objects containing all of the values the given request parameter has, or null if the parameter does not exist.

27. `boolean isSecure()`

Returns a boolean indicating whether this request was made using a secure channel, such as HTTPS.

28. `int getContentLength()`

Returns the length, in bytes, of the request body and made available by the input stream, or -1 if the length is not known.

29. `int getIntHeader(String name)`

Returns the value of the specified request header as an int.

30. `int getServerPort()`

Returns the port number on which this request was received.

HTTP Header Request Example

Following is the example which uses `getHeaderNames()` method of `HttpServletRequest` to read the HTTP header information. This method returns an Enumeration that contains the header information associated with the current HTTP request.

Once we have an Enumeration, we can loop down the Enumeration in the standard manner. We will use the *`hasMoreElements()`* method to determine when to stop and the *`nextElement()`* method to get the name of each parameter name.

```

<%@ page import = "java.io.*,java.util.*" %>

<html>
  <head>
    <title>HTTP Header Request Example</title>
  </head>

  <body>
    <center>
      <h2>HTTP Header Request Example</h2>

      <table width = "100%" border = "1" align = "center">
        <tr bgcolor = "#949494">
          <th>Header Name</th>
          <th>Header Value(s)</th>
        </tr>
        <%
          Enumeration headerNames = request.getHeaderNames();
          while(headerNames.hasMoreElements()) {
            String paramName = (String)headerNames.nextElement();
            out.print("<tr><td>" + paramName + "</td>\n");
            String paramValue = request.getHeader(paramName);
            out.println("<td> " + paramValue + "</td></tr>\n");
          }
        %>
      </table>
    </center>
  </body>
</html>

```



T

Background of Servlet

A Servlet is a Java program that runs on a web server and handles requests from web clients (usually browsers). It was introduced to overcome the limitations of traditional web technologies and to provide a powerful, secure, and portable server-side programming solution.

Before Servlets: Early Web Technologies

Before Servlets, server-side programming relied mainly on:

(a) CGI (Common Gateway Interface)

Programs written in languages like C, Perl, Python, etc. A new process was created for every request was very slow. Not platform-independent. Hard to maintain and not scalable.



(b) Server-side scripting Examples:

ASP, Perl scripts, etc. Depended on server and OS.
Not secure or efficient for large applications.

2. Need for Servlets

Java introduced Servlets to solve the problems of CGI: Why Servlets were needed?

- ✓ Platform Independent (Write Once, Run Anywhere)
- ✓ High Performance - servlet runs inside a single JVM
- ✓ Better Memory Management
- ✓ Security features (Java security model)
- ✓ Scalability
- ✓ Reusable and maintainable (Object-oriented)

Servlets became part of Java EE (Enterprise Edition) for building dynamic web applications.



3. Servlets and Web Containers

Servlets run inside a Servlet Container or Web Container, such as:

1. Apache Tomcat
2. Jetty
3. IBM
4. WebSphere
5. Oracle
6. WebLogic

The container:

Manages servlet lifecycle

Handles request/response objects

Provides security, session management, etc.

4. Evolution Toward Modern Java Web Technologies

Servlets provided the foundation for:

1. JSP (Java Server Pages)
2. JSF (Java Server Faces)
3. Spring MVC
4. Struts Framework



Life Cycle of a Servlet

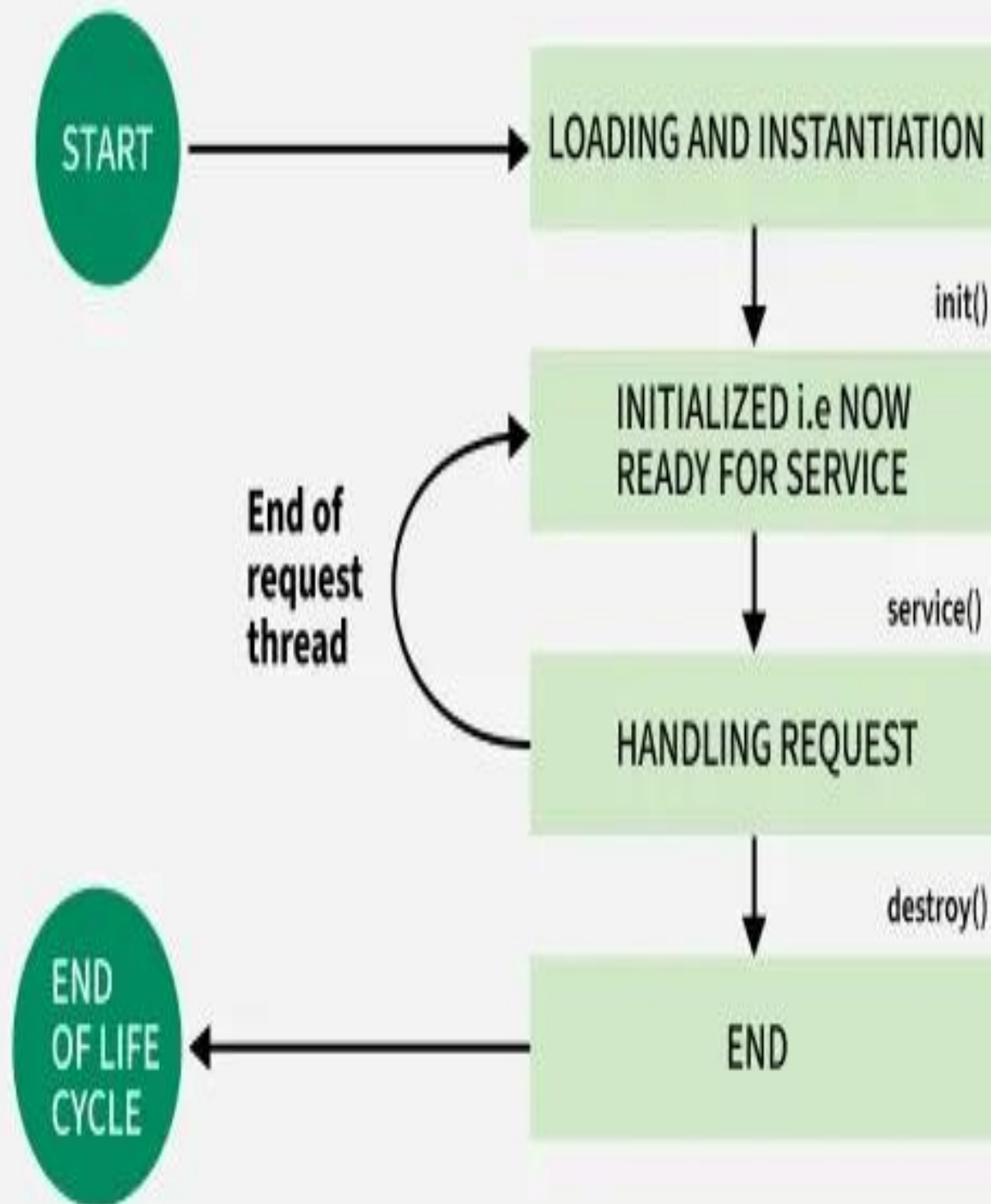
A Servlet is a Java program that runs on a server to handle client requests and generate dynamic responses. The Servlet life cycle is entirely managed by the Servlet container, which uses the `jakarta.servlet.Servlet` interface to manage Servlet objects.

Understanding the life cycle helps in writing efficient and resource-safe web applications.

The Servlet life cycle consists of four main stages:

1. Loading a Servlet
2. Initializing a Servlet
3. Handling Client Requests
4. Destroying a Servlet

State of sevlet life cycle





1. Loading a Servlet

The first stage of the Servlet lifecycle involves loading and initializing the Servlet. The Servlet container performs the following operations:

Loading: The Servlet container loads the Servlet class into memory.

Instantiation: The container creates an instance of the Servlet using the no-argument constructor.

When is a Servlet loaded?

During application startup (if configured in deployment descriptor).

On first client request (lazy loading).

2. Initializing a Servlet

After instantiation, the container initializes the Servlet using the `init()` method.

`init(ServletConfig config)` method is called once in the Servlet's lifetime.

It prepares the Servlet to handle requests (e.g., database connection setup).

Called only once, unlike `service()` which is called per request.

Example:

@Override

```
public void init() throws ServletException {  
    // Initialization code (e.g., database connection)  
}
```

3. Handling request

Once initialized, the Servlet is ready to handle client requests.

3.1 Request and Response Objects

ServletRequest and ServletResponse objects are created for each request.

For HTTP, HttpServletRequest and HttpServletResponse objects are used.

3.2 The service() Method

The container calls service(ServletRequest req, ServletResponse res) for each request.

Determines the HTTP request type (GET, POST, PUT, DELETE).

Delegates the request to appropriate methods: doGet(), doPost(), etc.

Example:

@Override

```
public void service(ServletRequest req, ServletResponse res)
    throws ServletException, IOException {
    // Handle request and generate response
}
```


Destroying a Servlet

When the Servlet container decides to remove the Servlet, it follows these steps which are listed below

Allow Active Threads to Complete: The container ensures that all threads executing the `service()` method complete their tasks.

Invoke the `destroy()` Method: The container calls the `destroy()` method to allow the Servlet to release resources (e.g., closing database connections, freeing memory).

Release Servlet Instance: After the `destroy()` method is executed, the Servlet container releases all references to the Servlet instance, making it eligible for garbage collection















